

Short Term Training Program on “GenAI” Assignment

Assignment-1:

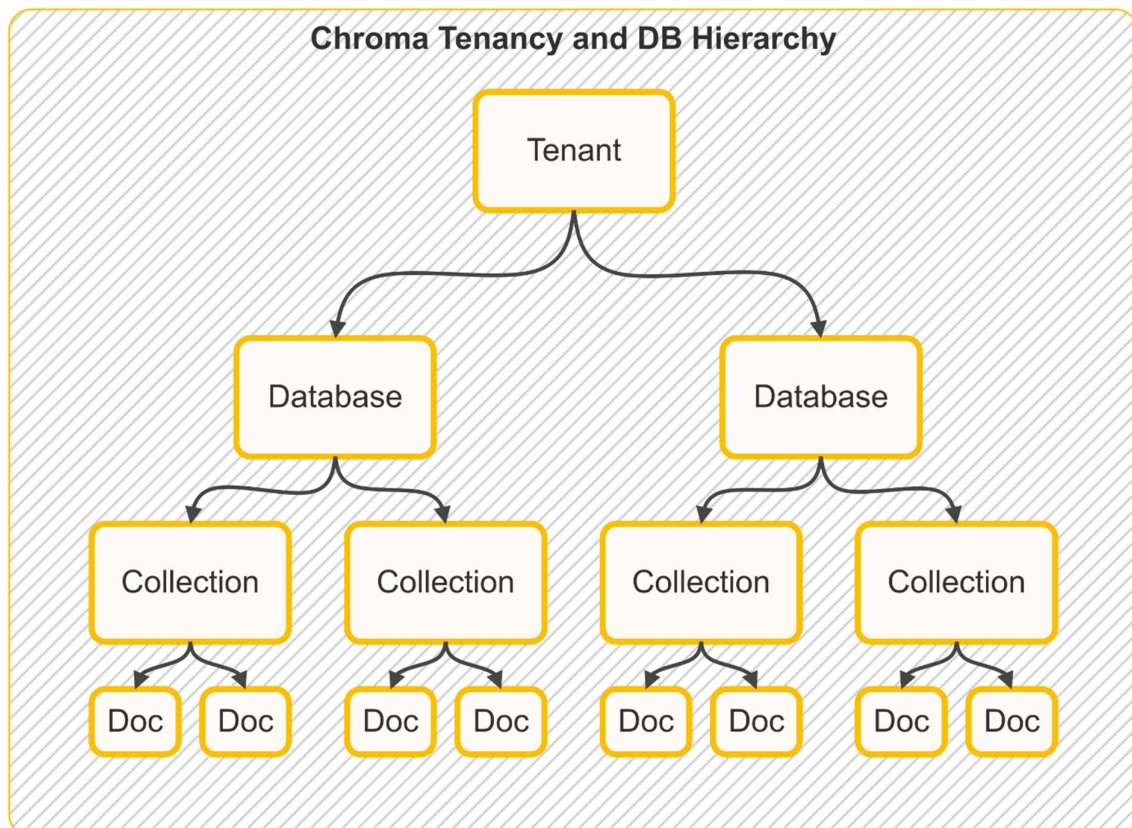
Research ChromaDB

- Architecture
- Features
- Use cases

Ans.

ChromaDB is an open-source, AI-native vector database to store, query, and manage embeddings alongside document text and metadata. It acts as the long-term memory layer for AI agents and Retrieval-Augmented Generation(RAG) pipelines.

Architecture:



Hierarchical Structure

ChromaDB organizes its data in a hierarchical manner to ensure flexibility and scalability:

- **Tenants:** Represent organizations or individuals using ChromaDB. Each tenant can manage multiple databases, enabling logical separation of data.
- **Databases:** Logical containers for collections, typically corresponding to a single application or project. Each database can house multiple collections.
- **Collections:** Groups of documents(data entries) that share similar characteristics. Collections organize embedding, documents, and metadata without requiring a predefined schema. A collection is the fundamental unit of storage and querying in Chroma. Each collection contains items with:
 - A unique ID
 - An embedding vector
 - Optional metadata
 - A document
- **Documents:** Raw chunks of text or data stored in ChromDB. Each document is associated with a vector embedding and optional metadata, enabling efficient querying and retrieval.

Deployment Modes:

ChromaDB supports multiple deployment modes to cater to different use cases:

- **Local Mode:** Embedded library for prototyping and experimentation.
- **Single Node Mode:** Suitable for small to medium workloads.
- **Distributed Mode:** Scalable for large-scale production workloads, supporting millions of collections. It uses cloud object storage for durability and local SSDs for caching to reduce latency.

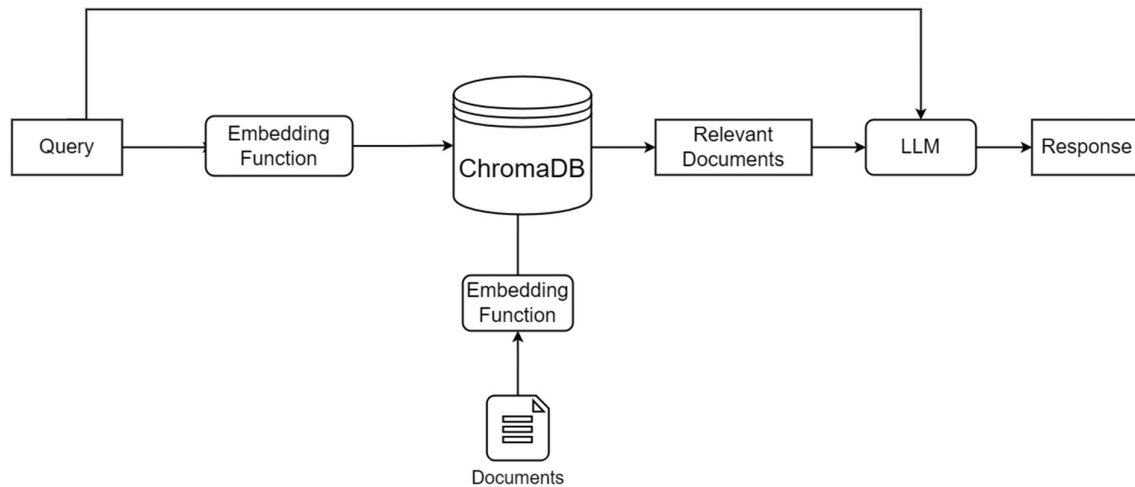
Features:

ChromaDB offers advanced capabilities such as:

- **Document Storage** – Store documents and metadata
- **Embeddings** – Compatible with OpenAI, Cohere, Hugging Face, sentence-transformers, and more
- **Vector Search** – Dense, sparse, and hybrid search strategies
- **Full-Text & Regex Search** – Keyword/regex search without needing embeddings
- **Metadata Filtering** – Filter query results by metadata conditions
- **Multi-Modal Retrieval** – Index and search images, audio, and other modalities alongside text
- **Open Source** – Apache 2.0 licensed; run locally, self-host, or use Chroma Cloud

Use Cases:

- **RAG (Retrieval-Augmented Generation)** – Ground LLMs in private/custom data by retrieving relevant chunks before prompting
- **Semantic Search** – Find documents by meaning, not just keywords
- **Hybrid Search** – Combine dense (semantic) + sparse (keyword) search for technical/specialized content
- **Agentic AI Systems** – Power AI agents that plan, search, evaluate, and iterate across multiple collections
- **Question Answering** – Build Q&A systems over knowledge bases
- **Legal Assistants** – Search across case law, statutes, and regulatory documents
- **Medical AI** – Query clinical guides, research papers, and patient records
- **Customer Support Bots** – Navigate product docs, past tickets, and knowledge bases
- **Coding Assistants** – Search documentation, code repos, and issue trackers

Assignment-2:**Design Memory Flow Diagram****Ans.**

- **Documents:** The raw knowledge base (PDFs, logs, wikis) parsed into smaller text chunks.
- **Embedding Function:** The translation model that converts raw text—both source documents and incoming user queries—into mathematical vectors.
- **ChromaDB:** The vector database that indexes the embeddings and executes rapid similarity searches to find matching data.
- **Query:** The user's live input string, which simultaneously kicks off the database vector search and routes directly to the LLM.
- **Relevant Documents:** The highly targeted context snippets retrieved by ChromaDB that contain the answers to the user's query.
- **LLM:** The generative brain that ingests both the raw query and the retrieved context to synthesize an articulate answer.
- **Response:** The finalized, factually grounded output delivered back to the user with minimized risk of hallucination.

Assignment-3**Compare Traditional Search vs Semantic Search****Ans.**

Dimension	Traditional Search (Lexical)	Semantic Search (Vector)
Core Mechanism	Matches exact alphabetical characters, strings, or token frequencies.	Matches underlying conceptual meaning, intent, and context.
Mathematical Basis	Statistical frequency algorithms like TF-IDF or BM25.	Distance metrics in high-dimensional vector space like Cosine Similarity or Dot Product.
Handling Synonyms	Fails unless manually mapped (e.g., treats "automobile" and "car" as completely different).	Understands synonyms natively because they map closely together in the vector space.
Query Flexibility	Relies on strict keywords, boolean operators (AND, OR), and wildcards (*).	Handles full natural language sentences, conversational questions, and typos smoothly.
Supported Data	Limited primarily to text and structured database fields.	Highly flexible; handles text, code, images, audio, and video (multimodal).
Resource Cost	Low CPU & RAM footprint. Indexing is static and query lookups are computationally cheap.	High CPU, GPU, & RAM footprint. Requires ML models for inference and large RAM for graph structures (like HNSW).
Example Technologies	Elasticsearch, Apache Solr, standard SQL Full-Text Search.	ChromaDB, Pinecone, Milvus, Qdrant, pgvector.